

TABLE PARTITIONING IN SQL SERVER

Name	Sultan Mamud
Designation	CAT1 Trainee
Section	ACS
Division	ACBDD
Section Head	Shri Rajesh Kumar Agrawal (Head, Accelerator Controls Section)
Coordinator	Smt. Bhavna Nitin Merh (Head, Software, Networking & Security Lab.)
Organization	RRCAT
Duration	02-09-2025 to 02-12-2025

Table of Contents

1. Objective:	3
2. Test Environment	4
2.1 Hardware and Cluster Setup	4
2.1.1 Domain Overview	4
2.1.2 Server Configuration Summary	4
2.1.3 Implementation Notes	5
2.2 Data Logging (ETL) Simulation – High Level Overview	5
2.3 Partition Switching and Archival Setup — High-Level Overview	6
3. Test Cases & Key Findings:	8
Test Case 1: Inserting records with values before the first partition and after the last partition....	8
Test Case 2: Insert rows exactly at boundary values of partition function.....	8
Test Case 3: Inserting NULL values in the partition key column.....	8
Test Case 4: Query performance on partitioned table vs non partitioned table.....	8
Test Case 5: Removing data of an entire partition.....	9
Approach 1: Using ‘DELETE’ DML command.....	9
Approach 2: Using a particular partition number and ‘DELETE’ DML command.....	9
Comparison of Approach 1 and Approach 2.....	9
Approach 3: Fast Deletion Using Partition Switch-Out.....	10
Test Case 6: Modifying partition boundaries (MERGE & SPLIT).....	10
Case 1: MERGE Command – Combining partitions.....	11
Case 2: SPLIT Command - Adding new partition boundary.....	11
Test Case 7: Partitioning on Existing INDUS2_VACUUM database.....	11
Test Case 8: Table-Level Backup in SQL Server.....	12
Test Case 9: Dynamic Partition Creation using SQL Server Agent Job.....	13
Test Case 10: Partitioning using multiple different filegroups.....	13
Test Case 11: Removing partition on existing tables.....	14
4. Implementation strategy:	17
Step 1: Restore database from backup folder.....	18
Step 2: Prepare database for partitioning.....	19
Step 3: Create yearly partition function and scheme.....	20
Step 4: Create temporary staging tables.....	21
Step 5: Create partitioned clustered indexes on main tables.....	21
Step 6: Purge old data using partition switching.....	22
Step 7: Rebuild indexes and shrink database.....	23
Step 8: Create yearly retention backup.....	24
Execution timeline summary (INDUS2_VACUUM database).....	24
5. Conclusion & Key Insights.....	25
5.1. Significant Performance Gains.....	25
5.2. Strategic Design Recommendations.....	25
5.3. Operational Considerations for Production.....	25

1. Objective:

The primary objective of this study is to design, implement, and rigorously evaluate advanced SQL Server table-partitioning strategies integrated with Windows Server Failover Clustering (WSFC) and SQL Server Always On Availability Groups (AoAG). The research focuses on transforming the efficiency of large-scale daily and annual maintenance operations specifically data archival and deletion within high-availability database environments.

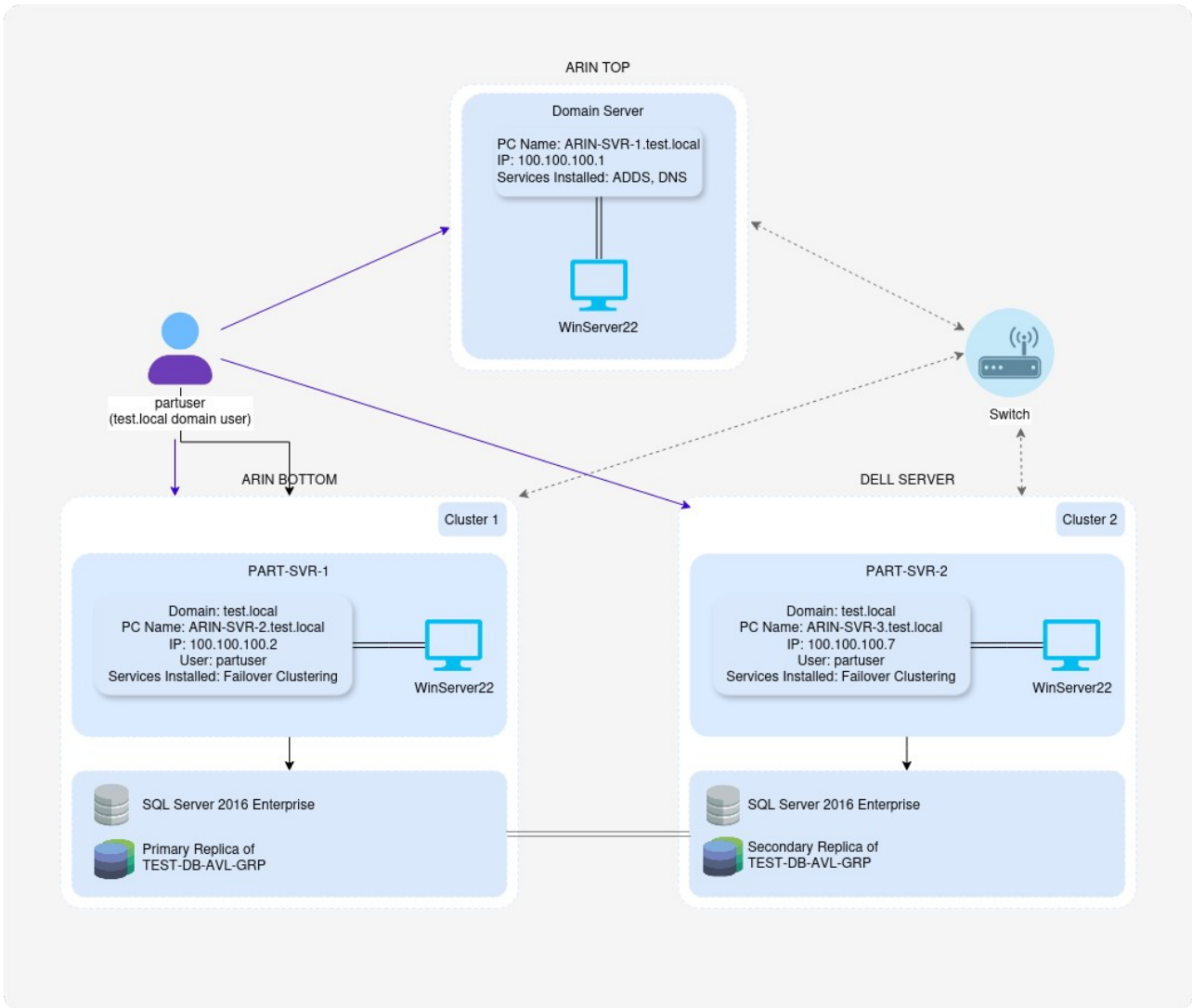
Conventional row-by-row deletion methods currently employed in the `INUDS2_VACUUM` database require approximately 1 to 3 hours for each annual purge cycle, generating substantial transaction log activity (If using Full recovery mode) and extended maintenance windows. This study proposes a partition-based data lifecycle management model that retains only the current year's data within staging tables while seamlessly migrating historical data to dedicated archival tables using metadata-only operations such as partition switching.

The proposed approach demonstrates a significant paradigm shift: while the initial migration of legacy data requires a comparable one-time execution window of approximately 2 to 3 hours, subsequent yearly maintenance operations are reduced to a matter of minutes. This results in negligible transaction log growth, minimal locking and blocking, and a dramatic reduction in system impact during maintenance windows. Additionally, the partitioned architecture enhances query performance for historical data and enables the creation of independent, year-specific backup files with minimal downtime.

All experiments are conducted within a controlled, non-production environment that mirrors enterprise-grade high-availability deployments. The infrastructure includes Active Directory Domain Services (ADDS) and DNS as foundational dependencies for Always On Availability Groups, ensuring the validity and portability of the findings across clustered SQL Server deployments. The outcomes of this research aim to provide a scalable, high-performance, and operationally efficient data management framework suitable for modern SQL Server environments requiring both high availability and long-term data retention.

2. Test Environment

2.1 Hardware and Cluster Setup



The test environment is configured within a single Active Directory domain named **test.local**. It consists of **three primary servers**, each serving distinct roles within the infrastructure.

2.1.1 Domain Overview

- **Domain Name:** test.local
- **Primary Services:** Active Directory Domain Services (AD DS), DNS, and Windows Server Failover Clustering
- **Operating System:** Windows Server 2022 (All Servers)

2.1.2 Server Configuration Summary

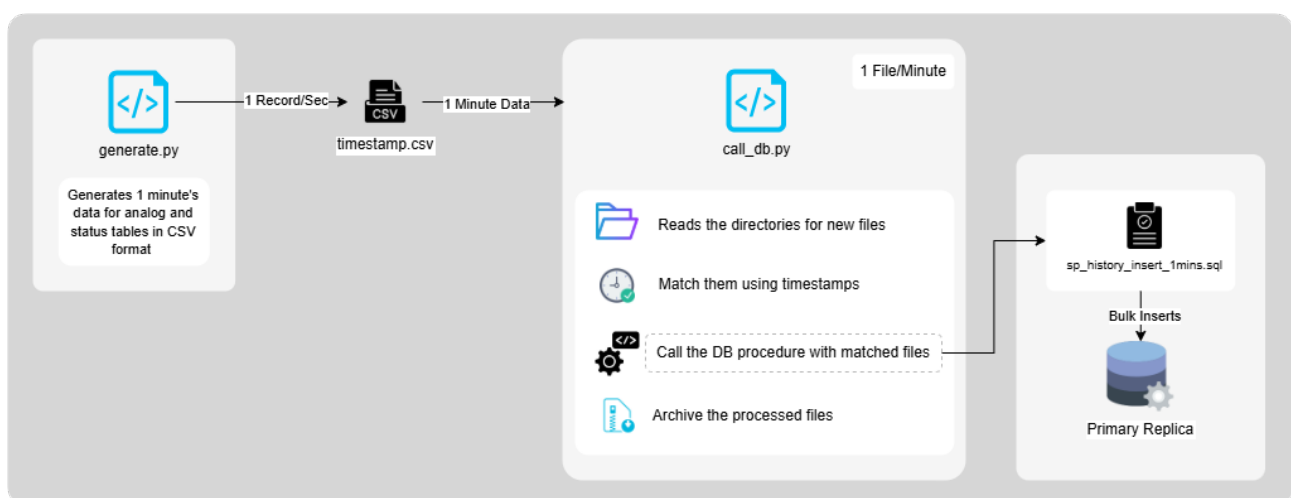
Server	PC Name	Domain	IP Address	CPU	RAM	Storage	Operating	Notes / Role
--------	---------	--------	------------	-----	-----	---------	-----------	--------------

Role	Membersh ip				System			Description
Domain Controller & DNS	ARIN-SVR-1	test.local	100.100.100.1	Intel Xeon Silver 4313 @ 2.4 GHz	128 GB	10.5TB (RAID5)	Windows Server 2022	Hosts Active Directory and DNS services
Cluster Node 1	ARIN-SVR-2	test.local	100.100.100.2	Intel Xeon Silver 4313 @ 2.4 GHz	128 GB	10.5TB (RAID5)	Windows Server 2022	Configured with Windows Failover Clustering; hosts database files
Cluster Node 2	ARIN-SVR-3	test.local	100.100.100.7	Intel Xeon Silver 4208 @ 2.1 GHz	128 GB	6TB (RAID5)	Windows Server 2022	Secondary cluster node; hosts database files

2.1.3 Implementation Notes

- All servers are members of the **test.local** domain and connected within the same subnet.
- **Failover Clustering** is configured between ARIN-SVR-2 and ARIN-SVR-3 to ensure high availability of database services.
- The **Domain Controller (ARIN-SVR-1)** provides authentication, DNS, and directory services for the environment.
- Each system is configured with **redundant storage (RAID 5)** to support data reliability and performance.

2.2 Data Logging (ETL) Simulation – High Level Overview



This setup automates **real-time data simulation**, **CSV generation**, **SQL ingestion**, and **archival** for the TEST_DB SQL Server database. It consists of **two Python scripts** and one **SQL stored procedure** working in sequence to generate, store, and insert process data.

High-level Flow Summary:

1. Data Generator (generate_data()):

- Continuously produces **simulated analog and status data** every second.
- Data points:
 - 304 analog columns (floating-point values)
 - 576 status columns (binary values)
- Groups data into **1-minute batches (60 rows)**.
- Each batch is saved as two CSV files (vac_analog_*.csv and vac_status_*.csv) using the **timestamp of the first record** as the filename reference.

2. File Processor (process_files()) running continuously every 60 seconds):

- Scans both analog and status directories.
- Identifies **file pairs with matching timestamps** that are **older than 2 minutes** (to avoid partially written files).
- Calls the SQL Server stored procedure sp_history_insert_1mins for each matched pair.
- On successful insertion:
 - Moves the files into their respective **archive folders**.
 - Logging captures success, failure, or skipped operations.

3. SQL Stored Procedure (sp_history_insert_1mins):

- Performs **BULK INSERT** of the analog and status CSVs into:
 - TEST_DB.dbo.vacuum_analog
 - TEST_DB.dbo.vacuum_status
- Uses the same directories as defined in the Python script paths.
- Tracks and updates status/error codes for each operation.

2.3 Partition Switching and Archival Setup – High-Level Overview

To optimize query performance and streamline historical data management by implementing **daily table partitioning** and **automated partition switching** between staging and main tables in TEST_DB this setup is incorporated. Later in some test cases you may find that monthly or yearly partitioning strategy was followed due to various real life scenario in our production environment.

Partition Design

- **Partition Function:**
PF_LOGTIME_PER_DAY divides data **by logtime at daily intervals**.
Each partition corresponds to a specific date boundary (e.g., 2025-10-01, 2025-10-02, etc.).
- **Partition Scheme:**
PS_LOGTIME_PER_DAY maps all partitions to the [PRIMARY] filegroup.
(Can be extended later to multiple filegroups for storage balancing.)
- **Clustered Indexing:**
All main and staging tables (vacuum_analog, vacuum_status, main_vacuum_analog, main_vacuum_status) are clustered on logtime and aligned to the partition scheme for **switch compatibility**.

Core Workflow

1. **Data Load Stage:**
 - Incoming one-minute data batches are first inserted into the **staging tables** (vacuum_analog, vacuum_status) via the ETL process.
2. **Partition Evaluation:**
 - Each partition represents one calendar day's worth of data.
 - Once a partition (day) is complete and new data moves to the next day, the closed partition becomes **eligible for switching**.
3. **Switch Operation:**
 - The PROC_SWITCH_PARTITION procedure:
 - Validates table existence and partition alignment.
 - Identifies eligible partitions (completed days, no overlapping data).
 - Performs an **ALTER TABLE ... SWITCH PARTITION** to move the entire day's data from the staging table to the main table **instantly (metadata-level operation)**.
4. **Logging and Monitoring:**
 - Each action (validation, eligibility check, success/failure) is recorded in JobSwitchLog for audit and troubleshooting.

Automation via SQL Agent Job

- **Job Name:** PARTITION_SWITCH_JOB_TEST_DB
- **Schedule:** Runs automatically (e.g., every 5 minutes or daily at midnight).
- **Action:** Executes PROC_SWITCH_PARTITION for both table pairs:
 - vacuum_analog → main_vacuum_analog
 - vacuum_status → main_vacuum_status
- **Execution Flow:**

- Checks eligible partitions.
- Switches closed partitions.
- Logs outcomes for each partition processed.

3. Test Cases & Key Findings:

Test Case 1: Inserting records with values before the first partition and after the last partition

Consider the below partition function

```
CREATE PARTITION FUNCTION PF_LOGTIME_PER_DAY (DATETIME2)
AS RANGE RIGHT FOR VALUES
(
    '2025-10-01 00:00:00',
    '2025-10-02 00:00:00',
    '2025-10-03 00:00:00'
);
```

Even though only three partition boundaries were mentioned but internally four partitions were created.

Partition 1: values < '2025-10-01 00:00:00'

Partition 2: values >= '2025-10-01 00:00:00' and values < '2025-10-02 00:00:00'

Partition 3: values >= '2025-10-02 00:00:00' and values < '2025-10-03 00:00:00'

Partition 4: values >= '2025-10-03 00:00:00'

Test Case 2: Insert rows exactly at boundary values of partition function.

Consider the above partition, suppose some table is utilizing the same partitioning strategy, and inserts a record with timestamp '2025-10-01 00:00:00'. In this scenario the record will be inserted in the second partition.

Test Case 3: Inserting NULL values in the partition key column

The insert will succeed. SQL Server always places the NULL values in partition 1 (the leftmost partition), regardless of whether we use RANGE LEFT or RANGE RIGHT.

Test Case 4: Query performance on partitioned table vs non partitioned table.

Non partitioned INDUS2_VACUUM

```
SELECT COUNT(*)
FROM main_vacuum_analog
WHERE logtime BETWEEN '2023-05-01' and '2023-07-01'
```

Execution time: 1:14 mm:ss

Partitioned INDUS2_VACUUM

```
SELECT COUNT(*)
FROM main_vacuum_analog
WHERE logtime BETWEEN '2023-05-01' and '2023-07-01'
```

Execution time: 2s

Test Case 5: Removing data of an entire partition

In this scenario, we explored various strategies to find out the most optimized way.

Approach 1: Using 'DELETE' DML command

```
SET STATISTICS TIME ON;
DELETE FROM main_vacuum_analog
WHERE DAY(logtime) = 03;
SET STATISTICS TIME OFF;
```

Result:

CPU time = 1719 ms, elapsed time = 7470 ms.
(86400 rows affected)

Approach 2: Using a particular partition number and 'DELETE' DML command.

```
SET STATISTICS TIME ON;
DELETE FROM main_vacuum_analog
WHERE $partition.PF_LOGTIME_PER_DAY(logtime) = 5;
SET STATISTICS TIME OFF;
```

Result:

CPU time = 469 ms, elapsed time = 5313 ms.
(86400 rows affected)

Comparison of Approach 1 and Approach 2

-- Using Date Range

```
SET STATISTICS TIME ON;
DELETE FROM main_vacuum_analog
WHERE CAST(logtime AS DATE) BETWEEN '2023-01-01' AND '2023-01-31';
```

```

SET STATISTICS TIME OFF;

-- Using Partition Function
SET STATISTICS TIME ON;
DELETE FROM main_vacuum_analog
WHERE $partition.PF_LOGTIME_PER_MONTH(logtime) = 3;
SET STATISTICS TIME OFF;

```

Result:

```

CPU time = 24,920 ms, elapsed time = 45,469 ms. (2,678,369 rows)
CPU time = 14,422 ms, elapsed time = 331,172 ms. (2,267,287 rows)

```

Observation:

Execution time depends on data volume.

Approach 3: Fast Deletion Using Partition Switch-Out

Step 1: Create a helper table (with same structure & partitioning strategy)

```

CREATE TABLE [dbo].[temp_main_vacuum_analog](
    [logtime] [datetime] NULL,
    [st1_vs1_bag1_rb] [real] NULL,
    [st1_vs1_bag2_rb] [real] NULL,
    [st1_vs1_bag3_rb] [real] NULL,
    [st1_vs1_bag4_rb] [real] NULL
);

CREATE CLUSTERED INDEX CIX_temp_main_vac_analog
ON temp_main_vacuum_analog(logtime)
ON PS_LOGTIME_PER_DAY(logtime);

```

Step 2: Switch partition using ALTER TABLE ... SWITCH PARTITION

```

ALTER TABLE dbo.main_vacuum_analog
SWITCH PARTITION 6
TO dbo.temp_main_vacuum_analog PARTITION 6;

```

Step 3: Truncate switched out data

```

TRUNCATE TABLE temp_main_vacuum_analog;

```

Observation: switch partition is a metadata command and truncate command is DDL command thus increasing the performance compared to DML command and its almost instantaneous.

Test Case 6: Modifying partition boundaries (MERGE & SPLIT)

Objective is to test the commands of modifying partition boundaries.

Case 1: MERGE Command – Combining partitions

Command:

```
ALTER PARTITION FUNCTION PF_LOGTIME_PER_DAY()  
MERGE RANGE (<BoundaryValue>);
```

Observation 1:

SQL Server's **ALTER PARTITION FUNCTION ... MERGE RANGE** command, which is used to merge partitions, is not always a metadata-only operation. Its performance depends significantly on the data volume of the involved partitions. When the consecutive partitions are empty the MERGE operation does not involve any physical data movement thus provides a near instantaneous execution. But in case of non empty partitions the physical data movement from one partition to another requires heavy consumption of resources depending of volume of data, row size and presence and alignment of indexes.

Observation 2:

You can only merge two adjacent partitions at a time. SQL Server does not support merging multiple partition at a time.

Case 2: SPLIT Command - Adding new partition boundary

Command:

```
SET STATISTICS TIME ON  
ALTER PARTITION SCHEME PS_LOGTIME_PER_DAY  
NEXT USED [PRIMARY];  
  
ALTER PARTITION FUNCTION PF_LOGTIME_PER_DAY()  
SPLIT RANGE (<BOUNDARY VALUE>)  
SET STATISTICS TIME OFF
```

Observation:

Similar to the MERGE command SPLIT RANGE command is also a hybrid operation that is instantaneous metadata level for new empty partitions but can be heavy, if any data movement is required. For example, if you are splitting a partition having existing data the performance of this command will depend on the volume of data.

Test Case 7: Partitioning on Existing INDUS2_VACUUM database

Step 1: Drop existing indexes.

Step 2: Drop old Partition Function (PF) and Partition Scheme (PS).

Step 3: Create new PF and PS.

Step 4: Recreate indexes on the new PF and PS.

Execution Summary of creating indexes:

Start Time : 13:35:00 (approx)

```
Completion : 16:59:45 (+05:30)
```

```
CIX_main_analog: CPU 2,532,189 ms | Elapsed 11,160,949 ms
```

```
CIX_analog      : CPU 30,857 ms   | Elapsed 69,497 ms
```

```
CIX_main_status: CPU 3,656,111 ms | Elapsed 2,375,677 ms
```

```
CIX_status      : CPU 27,731 ms   | Elapsed 22,436 ms
```

Test Case 8: Table-Level Backup in SQL Server

Option 1: Export Data via SSMS

Option 2: Generate Script with Data via SSMS

Option 3: Copy Table Between Databases

```
SELECT * INTO backupdb.dbo.table  
FROM originaldb.dbo.table;
```

Option 4: Fastest – Using **bcp** Command

Backup:

```
bcp INDUS2_VACUUM.dbo.temp_main_vacuum_analog out "C:  
\Users\partuser\Downloads\temp_main_vac_analog_2024.bak" -n -S ARIN-SVR-2 -T
```

Parameters:

-n → Native format (fastest)

-S → SQL Server instance name

-T → Windows Authentication

Result:

```
31,144,283 rows copied.  
Total time: 898,719 ms (~15 minutes)  
Average: 34,654 rows/sec
```

Restore:

```
bcp INDUS2_VACUUM_TEST.dbo.main_vacuum_analog in "C:  
\Users\partuser\Downloads\temp_main_vac_analog_2024.bak" -n -S ARIN-SVR-2 -T
```

Result:

```
31,144,283 rows copied.  
Total time: 1,923,312 ms (~32 minutes)  
Average: 16,193 rows/sec
```

Test Case 9: Dynamic Partition Creation using SQL Server Agent Job

We can create partitions dynamically by a scheduled SQL Server Agent job.

We can use the SPLIT RANGE command in SQL Server.

```
BEGIN TRY
    ALTER PARTITION SCHEME PS_LOGTIME_PER_DAY NEXT USED [PRIMARY];
    SET @SQL_QUERY = N'ALTER PARTITION FUNCTION PF_LOGTIME_PER_DAY()
        SPLIT RANGE (''' + CONVERT(NVARCHAR(10), @NEW_BOUNDARY_DATE, 120)
+ N''');';
    EXEC sp_executesql @SQL_QUERY;
END TRY
```

Since we have already seen that creating new empty partitions is just a metadata operation, thus it does not require any significant time.

Test Case 10: Partitioning using multiple different filegroups

Objective: Evaluate partition switching performance and data management when partitions are mapped to separate filegroups. However, our initial objective was to perform filegroup level backup and restoration. But due to database consistency errors we were not able to perform the restorations.

Procedure:

```
-- Step 1: Drop existing indexes
DROP INDEX CIX_main_analog ON main_vacuum_analog;
DROP INDEX CIX_main_status ON main_vacuum_status;

-- Step 2: Create yearly partition scheme on filegroups
CREATE CLUSTERED INDEX CIX_main_analog
    ON main_vacuum_analog(logtime)
    ON PS_LOGTIME_YEARLY(logtime); -- Maps to FG_2023, FG_2024, etc.

CREATE CLUSTERED INDEX CIX_main_status
    ON main_vacuum_status(logtime)
    ON PS_LOGTIME_YEARLY(logtime);

-- Step 3: Switch partition to staging table
ALTER TABLE main_vacuum_analog
    SWITCH PARTITION 3 TO stg_main_vacuum_analog PARTITION 3;

-- Step 4: Extract switched data
SELECT * INTO INDUS2_VAC_23.dbo.stg_main_vacuum_analog
FROM INDUS2_VACUUM.dbo.stg_main_vacuum_analog;
```

Results:

- **Index creation:** 2,922 seconds (CPU: 872,936 ms)
- **Partition switch:** < 1 second (metadata operation)
- **Data extraction:** 284 seconds for 23,098,906 rows (81,300 rows/sec)
- **Filegroup backup:** Processed 14,090,791 pages in 3,367 seconds

Observations:

Filegroup-level backup succeeded but restoration failed due to database consistency errors. Partition switching remained instantaneous regardless of filegroup configuration. Data extraction performance depended primarily on I/O throughput rather than partitioning strategy.

Conclusion: Partition switching provides near-instant data movement even with multiple filegroups; however, filegroup backup/restore operations require additional testing and validation for production use.

Test Case 11: Removing partition on existing tables

To remove partitioning from a table and clean up the associated metadata in SQL Server, you must follow a strict reverse-dependency order.

- You **cannot** drop a **Partition Function** if a **Partition Scheme** is using it.
- You **cannot** drop a **Partition Scheme** if a **Table or Index** is using it.

Following the correct sequence avoids errors and ensures a clean metadata state.

The Cleanup Workflow

1. **Unbind the Table**
Rebuild the clustered index onto a standard filegroup (for example, PRIMARY).
2. **Drop the Partition Scheme**
Remove the metadata linking the partition function to filegroups.
3. **Drop the Partition Function**
Remove the logical boundary definitions entirely.

Step-by-Step Execution Guide

Step 1: Rebuild the Clustered Index on a Single Filegroup

Partitioning in SQL Server is physically implemented through the **clustered index**.

To “flatten” the table, rebuild the clustered index so that it resides on a regular filegroup instead of a partition scheme.

```
-- This rebuild moves the table off the partition scheme
```

```
-- and onto the PRIMARY filegroup
```

```
CREATE CLUSTERED INDEX [PK_YourTableName]  
ON [dbo].[YourTableName] ([YourPartitionKeyColumn])
```

```
WITH (DROP_EXISTING = ON)
ON [PRIMARY];
GO
```

Notes:

- This operation removes partitioning while preserving the clustered index.
- If you want to remove the clustered index entirely after unbinding:

```
DROP INDEX [PK_YourTableName]
ON [dbo].[YourTableName];
```

Step 2: Drop the Partition Scheme

Once **no tables or indexes** reference the partition scheme, it can be safely removed.

```
DROP PARTITION SCHEME [YourPartitionSchemeName];
```

Expected Result:

- The database no longer has any filegroup-to-partition-function mapping.
- All dependent objects must already be unbound, or this step will fail.

Step 3: Drop the Partition Function

After the partition scheme is removed, the partition function itself is no longer required and can be dropped.

```
DROP PARTITION FUNCTION [YourPartitionFunctionName];
```

Expected Result:

- All partition boundary definitions are removed.
- The database metadata is fully cleaned of partitioning objects.

Important Considerations

- **Data Size**
Rebuilding a large partitioned table onto a single filegroup requires sufficient free space on the target filegroup (for example, PRIMARY) to hold the entire table during the operation.
- **Locking and Downtime**
Rebuilding a clustered index is a blocking operation by default.
If you are using **Enterprise Edition**, you can reduce downtime by using:

```
WITH (DROP_EXISTING = ON, ONLINE = ON)
```

- **Transaction Log Impact**
In FULL recovery mode, this operation can generate significant log activity.

Consider switching temporarily to SIMPLE recovery if appropriate and allowed by your recovery strategy.

Final State Verification (Optional)

To confirm that partitioning has been fully removed:

SELECT

```
t.name AS TableName,  
i.name AS IndexName,  
ps.name AS PartitionScheme  
FROM sys.tables t  
JOIN sys.indexes i ON t.object_id = i.object_id  
LEFT JOIN sys.partition_schemes ps ON i.data_space_id = ps.data_space_id  
WHERE t.name = 'YourTableName';
```

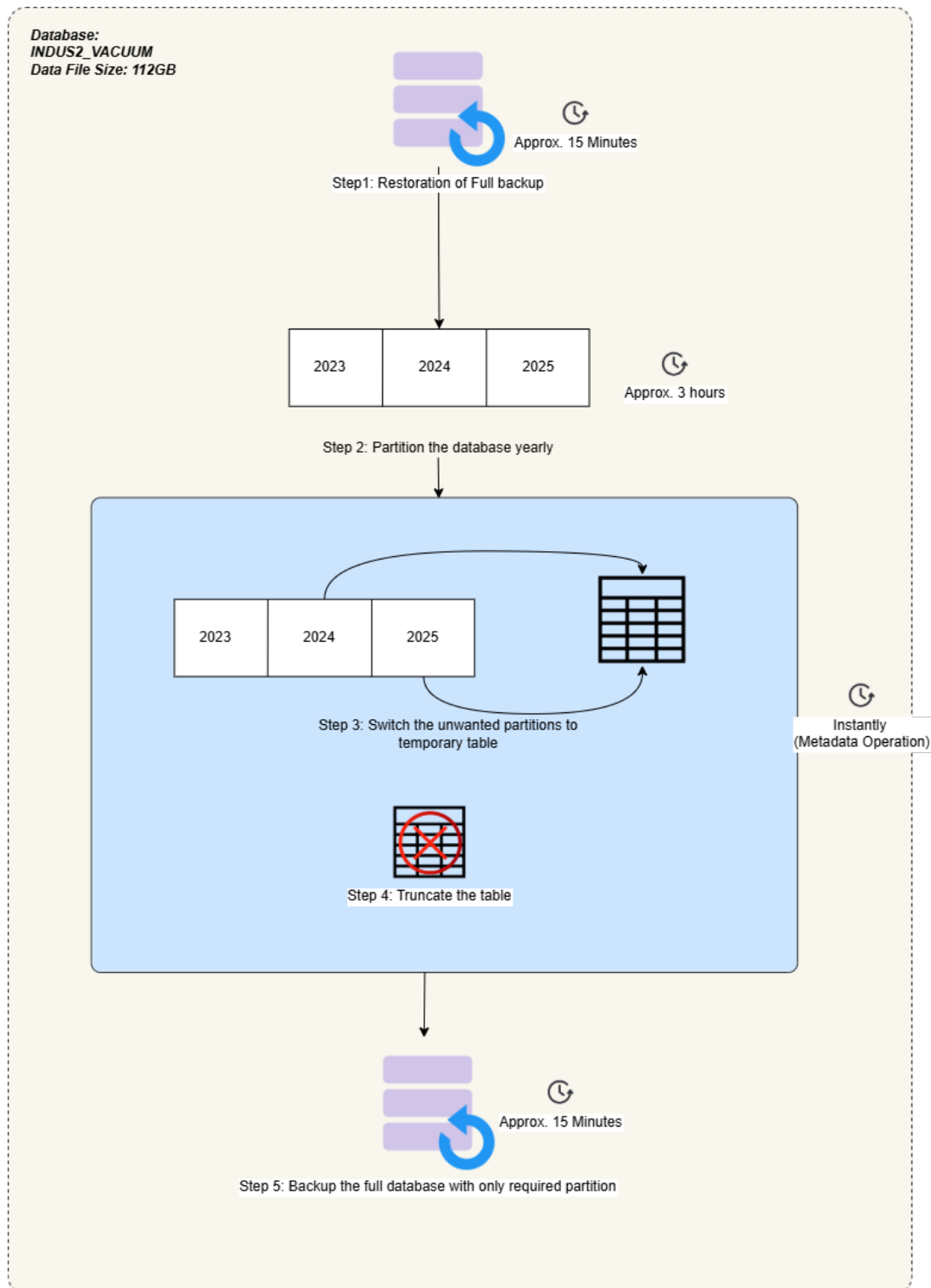
Result:

PartitionScheme should return **NULL**, confirming the table is no longer partitioned.

4. Implementation strategy:

This part of the document describes a complete, end-to-end strategy for implementing table partitioning in an existing high-volume SQL Server database i.e. INDUS2_VACUUM. The approach minimizes downtime, preserves data integrity, and enables fast historical data purging using partition switching.

Here's the high level overview of the process represented diagrammatically.



Step 1: Restore database from backup folder

Objective

Restore the database to a known, clean baseline using existing full and transaction log backups.

Procedure

1. Scan the backup directory for all .bak (full) and .trn (transaction log) files.
2. Read and validate backup headers to identify backup type, checkpoint LSN, and media set details.

Command example:

```
RESTORE HEADERONLY  
FROM DISK = 'E:\Backups\YourBackupFile.bak';
```

3. Select the optimal restore chain:

- Choose the most recent full backup (highest checkpoint LSN).
- Append all subsequent transaction log backups in correct LSN order.

4. Execute the restore sequence.

Full restore:

```
RESTORE DATABASE YourDatabase  
FROM DISK = 'E:\Backups\FullBackup.bak'  
WITH  
    MOVE 'YourLogicalDataName' TO 'D:\Data\YourDatabase.mdf',  
    MOVE 'YourLogicalLogName' TO 'D:\Logs\YourDatabase.ldf',  
    REPLACE,  
    NORECOVERY,  
    STATS = 5;
```

Restore transaction logs (repeat for each log file except last):

```
RESTORE LOG YourDatabase  
FROM DISK = 'E:\Backups\LogBackup.trn'  
WITH NORECOVERY;
```

Final log restore:

```
RESTORE LOG YourDatabase  
FROM DISK = 'E:\Backups\LastLogBackup.trn'  
WITH RECOVERY;
```

5. Monitor restore progress:

```
SELECT
```

```
percent_complete,  
estimated_completion_time / 1000 / 60 AS minutes_remaining  
FROM sys.dm_exec_requests  
WHERE command LIKE 'RESTORE%';
```

6. Verify database state:

```
SELECT name, state_desc  
FROM sys.databases  
WHERE name = 'YourDatabase';
```

Performance Notes

- Use STATS = 5 to monitor progress.
- If the database remains in RESTORING state, execute:

```
RESTORE DATABASE YourDatabase WITH RECOVERY;
```

- Set client connection timeout to infinite for large restores.

Step 2: Prepare database for partitioning

Objective

Reduce logging overhead and remove conflicting clustered indexes before partitioning.

Procedure

1. Switch recovery model to SIMPLE:

```
ALTER DATABASE YourDatabase SET RECOVERY SIMPLE;
```

2. Identify non-primary-key clustered indexes to drop:

```
SELECT  
  
    'DROP INDEX [' + i.name + '] ON [' +  
    SCHEMA_NAME(t.schema_id) + '].[' + t.name + ']' AS DropCommand  
FROM sys.tables t  
JOIN sys.indexes i ON i.object_id = t.object_id  
WHERE i.type_desc = 'CLUSTERED'  
    AND i.is_primary_key = 0  
    AND t.name LIKE '%main%';
```

3. Execute the generated DROP INDEX statements.

Expected Result

- All non-primary-key clustered indexes removed from main history tables.
- Database operating in SIMPLE recovery mode.

Step 3: Create yearly partition function and scheme

Objective

Define logical yearly partitions based on a datetime column.

Procedure

1. Drop existing partition objects if re-running:

```
IF EXISTS (SELECT 1 FROM sys.partition_schemes WHERE name =
'PS_LOGTIME_YEARLY')

DROP PARTITION SCHEME PS_LOGTIME_YEARLY;

IF EXISTS (SELECT 1 FROM sys.partition_functions WHERE name =
'PF_LOGTIME_YEARLY')
DROP PARTITION FUNCTION PF_LOGTIME_YEARLY;
```

2. Create the partition function (RANGE RIGHT):

```
CREATE PARTITION FUNCTION PF_LOGTIME_YEARLY (DATETIME2)
AS RANGE RIGHT
FOR VALUES
(
    '2022-01-01 00:00:00',
    '2023-01-01 00:00:00',
    '2024-01-01 00:00:00',
    '2025-01-01 00:00:00',
    '2026-01-01 00:00:00'
);
```

3. Create the partition scheme:

```
CREATE PARTITION SCHEME PS_LOGTIME_YEARLY
AS PARTITION PF_LOGTIME_YEARLY
ALL TO ([PRIMARY]);
```

Design Notes

- RANGE RIGHT places boundary values into the newer partition.
- NULL or pre-2022 values go to partition 1. Ensure application logic prevents NULL partition keys.

Step 4: Create temporary staging tables

Objective

Create partition-aligned staging tables for partition switching.

Procedure

1. Create a temp table with identical schema:

```
CREATE TABLE dbo.temp_main_vacuum_analog
(
    logtime DATETIME2 NOT NULL,
    st1_vs1_bag1_rb REAL NULL,
    st1_vs1_bag2_rb REAL NULL
    -- additional columns
);
```

2. Create a clustered index aligned to the partition scheme:

```
CREATE CLUSTERED INDEX CIX_temp_main_vacuum_analog
ON dbo.temp_main_vacuum_analog (logtime)
ON PS_LOGTIME_YEARLY (logtime);
```

Critical Requirement

Main and temp tables must match **exactly** in:

- Column definitions
- Data types
- NULL settings
- Clustered index definition
- Partition scheme

Step 5: Create partitioned clustered indexes on main tables

Objective

Physically partition the existing data by recreating clustered indexes.

Procedure

1. Identify tables without clustered indexes:

```
SELECT t.name
FROM sys.tables t
WHERE NOT EXISTS
(
```

```

SELECT 1
FROM sys.indexes i
WHERE i.object_id = t.object_id
      AND i.type_desc = 'CLUSTERED'
)
AND t.name LIKE '%main%';

```

2. Create clustered indexes on the partition scheme:

```

CREATE CLUSTERED INDEX CIX_main_vacuum_analog
ON dbo.main_vacuum_analog (logtime)
ON PS_LOGTIME_YEARLY (logtime);

```

3. Monitor execution time:

- Small tables: minutes
- Large tables (10M+ rows): hours
- CPU and I/O intensive

Data Distribution

- logtime < 2022-01-01 -> Partition 1
- 2022 -> Partition 2
- 2023 -> Partition 3
- 2024 -> Partition 4
- 2025 -> Partition 5
- 2026+ -> Partition 6

Step 6: Purge old data using partition switching

Objective

Instantly remove historical data without row-by-row deletes.

Procedure

1. Identify partition number:

```

DECLARE @PartitionID INT;

SELECT @PartitionID = $PARTITION.PF_LOGTIME_YEARLY('2022-01-01');

```

2. Switch the partition to the temp table:

```

ALTER TABLE dbo.main_vacuum_analog
SWITCH PARTITION @PartitionID

```

```
TO dbo.temp_main_vacuum_analog PARTITION @PartitionID;
```

3. Truncate the temp table:

```
TRUNCATE TABLE dbo.temp_main_vacuum_analog;
```

4. Repeat for all tables and all years to be purged.

Benefits

- Metadata-only operation
- Near-instant execution
- No transaction log bloat
- Minimal locking

Step 7: Rebuild indexes and shrink database

Objective

Optimize performance and reclaim disk space.

Procedure

1. Rebuild highly fragmented clustered indexes:

```
DECLARE @SQL NVARCHAR(MAX) = N'';
```

```
SELECT @SQL = @SQL +  
    'ALTER INDEX ALL ON ' +  
    QUOTENAME(SCHEMA_NAME(t.schema_id)) + '.' + QUOTENAME(t.name) +  
    ' REBUILD;' + CHAR(10)  
FROM sys.dm_db_index_physical_stats(DB_ID(), NULL, NULL, NULL, 'LIMITED')  
s  
JOIN sys.tables t ON s.object_id = t.object_id  
JOIN sys.indexes i ON s.object_id = i.object_id AND s.index_id =  
i.index_id  
WHERE s.avg_fragmentation_in_percent > 30  
AND i.type_desc = 'CLUSTERED';  
  
EXEC sp_executesql @SQL;
```

2. Shrink database:

```
DBCC SHRINKDATABASE (YourDatabase, 10);
```

3. Restore FULL recovery model:

```
ALTER DATABASE YourDatabase SET RECOVERY FULL;
```

Step 8: Create yearly retention backup

Objective

Maintain an isolated archival backup for the retained year.

Command

```
BACKUP DATABASE YourDatabase  
TO DISK = 'E:\Backups\YourDatabase_Year2025.bak'  
WITH FORMAT, COMPRESSION, STATS = 10;
```

Execution timeline summary (INDUS2_VACUUM database)

- **Full restore:** 15 to 30 minutes
- **Partition-based index creation:** 1 to 4 hours
- **Partition purge:** 5 to 10 minutes
- **Index rebuild and shrink:** 30 to 60 minutes
- **Backups:** 20 to 45 minutes

Total estimated duration: 2 to 5 hours (depending on database size).

Note: This estimate applies only to the initial implementation. In subsequent years, the partition-based index creation step will no longer be required, significantly reducing the overall execution time.

5. Conclusion & Key Insights

The evaluation of SQL Server table partitioning within a high-availability environment (WSFC and AoA) demonstrates that it is a highly effective strategy for managing large-scale, time-sensitive datasets. By shifting from row-based operations to metadata-level management, the system achieves significant improvements in both performance and maintainability.

5.1. Significant Performance Gains

Instant Data Movement: Utilizing `ALTER TABLE ... SWITCH PARTITION` allows for near-instant deletion or migration of large datasets. This is a metadata-level operation that avoids the transaction log bloat and resource heavy-lifting associated with standard `DELETE` commands.

Query Acceleration: Partitioning provides a dramatic boost to query efficiency for historical data. In testing, query execution times for count operations were reduced from 1 minute and 14 seconds on a non-partitioned table to just 2 seconds on a partitioned one.

5.2. Strategic Design Recommendations

Proactive Implementation: The study highlights that it is far more efficient to implement partitioning during the initial table design. Reconfiguring existing large tables for partitioning requires significant downtime for index recreation ranging from 1 to 4 hours depending on volume.

Logical Boundary Alignment: Using `RANGE RIGHT` is the most intuitive configuration for date-based data, as it ensures boundary values (like midnight) are correctly grouped with the day that follows.

Handling Boundary Edge Cases: It is critical for application logic to prevent or explicitly handle `NULL` values in the partition key, as SQL Server automatically defaults all `NULL` entries to the very first partition.

5.3. Operational Considerations for Production

Automation: To maintain a "set and forget" architecture, dynamic partition creation should be automated using SQL Server Agent jobs that execute `SPLIT RANGE` commands ahead of new time intervals.

Stability Concerns: While partitioning performs well with multiple filegroups, the observed consistency errors during filegroup-level restoration indicate that this specific workflow requires further validation and rigorous testing before being deployed in a production environment.

Maintenance Cycle: The initial implementation is the most resource-intensive. Subsequent maintenance such as purging old data becomes a routine, low-impact task taking only 5 to 10 minutes.